Team Riders

Point of Sale & Stock Control

https://www.teamriders.bid

Handbook for the shop team and the technical team

Issued 21 October 2026

Point of sale · Stock control · Profit tracking

Contents

1. The five-minute version . . . 5

2. Signing in, roles and safety . . . 5

2.1 Sign in and out . . . 5

2.2 What each role can do . . . 6

3. Selling at the counter . . . 6

3.1 The Point of Sale screen . . . 6

3.2 Discounts: proposed by the cashier, authorised by the admin . . . 7

3.3 Quotations . . . 7

3.4 Receipts: paper, PDF, or both . . . 7

3.5 Wrong sale? Void it — don't delete it . . . 8

3.6 Out of stock: capture the customer, then tell them when it lands . . . 8

4. Stock, purchasing and what the shop is worth . . . 8

4.1 Products . . . 8

4.2 Stock levels and reorder warnings . . . 8

4.3 Restocking, and paying for it out of the business . . . 8

4.4 Adjustments: breakage, theft, stock-take differences . . . 9

4.5 Breakages, returns and stock requests . . . 9

4.6 What the stock is worth . . . 9

5. Money, profit and the monthly close . . . 10

5.1 Business cash . . . 10

5.2 How profit is calculated . . . 10

5.3 Paying salaries . . . 10

5.4 The monthly close . . . 11

5.5 Reports and exports . . . 11

6. Admin settings and the audit trail . . . 11

6.1 Settings worth reviewing once . . . 12

6.2 Team & access . . . 12

6.3 Activity log . . . 12

6.4 Daily, weekly, monthly . . . 12

7. Technical overview . . . 13

7.1 Where things live . . . 13

7.2 How a request flows . . . 13

7.3 Repository layout . . . 14

8. Database, data model and integrity rules . . . 14

8.1 Tables . . . 15

8.2 Non-negotiable rules in the code . . . 15

8.3 How one backend flag changes everything . . . 16

9. Deployment, configuration and operations . . . 16

9.1 Configuration (environment variables) . . . 16

9.2 The change process . . . 16

9.3 Handy scripts . . . 17

9.4 Backups and restore . . . 17

9.5 Testing before and after a change . . . 18

9.6 Keeping the free instance awake . . . 18

10. Security, secrets and limits . . . 18

10.1 What protects the shop . . . 18

10.2 Secrets inventory . . . 19

10.3 Free-tier ceilings . . . 19

10.4 Known limitations . . . 19

11. Troubleshooting . . . 19

12. The offline fallback on the shop PC . . . 20

12.1 The Windows startup shortcut . . . 21

Appendix A — Route reference . . . 22

Appendix B — Settings keys . . . 22

Appendix C — Enumerations used by the data . . . 23

How to use this handbook
Part 1 is for the people behind the counter and in the office — it explains every screen in plain language. Part 2 is for whoever maintains the system (the shop's IT person or an outside developer) — it covers hosting, database, deployment, backups and fault-finding. Section 1 is the page to print and pin next to the till.

1. The five-minute version

Team Riders runs on the web at **www.teamriders.bid**. Any device with a browser — the shop laptop, a phone, a tablet — can use it, and it keeps working when the shop computer is switched off, because the application and the database live on the internet, not on the PC.

Question	Answer
Where do I open it?	https://www.teamriders.bid — bookmark it. `teamriders.bid` and `shop.teamriders.bid` also work.
How do I sign in?	Username and password issued by the owner. Cashiers only see the selling screens; the owner (admin) sees everything.
Who approves discounts?	The admin. A cashier can request one, but the sale cannot use it until it is approved.
Do I need to install anything?	No. No software on the PC, no card machine integration, no licence fees. A browser and internet are enough.
Where does the data live?	In a hosted PostgreSQL database (Neon), encrypted in transit. The shop PC holds no live copy unless you deliberately use the offline fallback.
What happens if the internet dies?	The shop PC can still run the offline copy (see 1.11 and 2.13) — but only one of the two may be used for real sales at a time.

Accounts for this handover
Administrator: username **LOYD**, temporary password **TR-Cloud-8842** — change it on first sign-in (Account -> Change password). A cashier account named **Brandon** exists; its password is not known to the technical team, so reset it from *Team & access*. Add or remove staff there, never by sharing one login.

2. Signing in, roles and safety

2.1 Sign in and out

1. Open **www.teamriders.bid** and enter your username and password.
2. You land on the dashboard. Cashiers see today's figures for their own selling; the admin sees the whole business.
3. Sign out with *Sign out* in the sidebar when you leave the counter — a session lasts 12 hours, sliding, so it renews while you work.
4. Forgotten password? Only the admin can reset it (Team & access -> user -> Reset password).

Five wrong passwords in a row lock that username for a short cooling-off period. This is deliberate: it stops someone guessing their way into the till. Wait and try again, or ask the admin to reset the password.

2.2 What each role can do

Capability	Cashier	Admin
Search prices, sell, print or save receipts	Yes	Yes
Create quotations	Yes	Yes
Capture a customer's e-mail when an item is out of stock	Yes	Yes
See their own receipts and reprint them (no cost, no profit)	Yes	Yes
Report a breakage, a customer return or a stock correction	Request only	Approve or refuse
See their own salary payments	Yes	Yes
See takings, profit, stock value, cash balance, cost prices	No	Yes
Give a discount	Request only	Approve or refuse
Add, edit, delete or archive products	No	Yes
Restock, adjust stock, read the stock ledger	No	Yes
Business cash ledger and paying salaries	No	Yes
Monthly close-off, reports, CSV exports	No	Yes
Team & access (create, disable, delete users)	No	Yes
Activity log and settings	No	Yes

Why cashiers cannot see profit
A cashier's screens deliberately show no takings, no profit, no stock value and no cash balance — not even as totals. They see the till, their own receipts, their own discount requests and the status of the breakages or returns they reported. Cost prices, margins and every business figure stay with the admin, and the server refuses those pages even if the address is typed by hand.

3. Selling at the counter

3.1 The Point of Sale screen

The till is **Point of Sale** in the sidebar. Everything below happens without leaving that page.

1. **Search** — start typing a name, part number, brand or bike model in the box at the top. A suggestion list opens as you type, showing SKU, name, **price** and how many units are in stock, so you can answer “how much is this?” without touching the till.
2. **Add to cart** — click a suggestion (or press Enter on the highlighted row). Repeat for each item; the cart on the right totals itself.
3. **Quantities** — change the quantity in the cart, or remove a line with the cross. Up to 999 units per line.
4. **Customer details (optional)** — name, phone and e-mail are stored on the sale and printed on the receipt; they also let the shop contact the customer later.
5. **Discount** — see 3.2. Leave it empty for a normal sale.
6. **Payment** — choose cash, mobile money, card or bank transfer. For cash, type what the customer handed over and the change is worked out for you.

7. **Complete sale** — stock is reduced, the receipt is numbered and you are taken to the receipt, ready to print.

What the system refuses to do
You cannot sell more units than the shelf holds — the till stops with “Not enough stock for …”. You cannot take less cash than the total. Nothing is ever sold at a negative price. These rules protect the stock count, so do not fight them: adjust the stock or ask the admin.

3.2 Discounts: proposed by the cashier, authorised by the admin

Step	What happens
1	On the sale, choose **percentage** or **fixed amount**, type the value and — mandatory — a **reason** (“loyalty”, “damaged box”, “price match”).
2	A cashier's request is created as **pending** with its own code (e.g. `DSC-15FFFC`). The sale cannot use it yet.
3	The admin opens *Discounts*, reads the reason and the amount, and approves or refuses with an optional note.
4	Once approved, the code is valid for **30 minutes** and is applied to the next sale that uses it.
5	After the sale the discount becomes **used** and is linked to that receipt. It cannot be reused.

House rules the system enforces: a reason is always required; a percentage discount may not exceed the limit in Settings (60% by default); an approved discount expires after 30 minutes; an unapproved request lapses after 24 hours. Unused, refused and expired attempts stay in the log, so discounting can always be audited.

When the admin is at the till: the admin can approve inline while selling, which issues an already-approved code valid for 30 minutes.

3.3 Quotations

1. Switch the till to **Quote** mode (the payment block disappears).
2. Search and add the parts exactly as for a sale, include customer details, apply an approved discount if the customer has one.
3. Save the quotation — it is numbered `QTN-YYYYMM-0001` and marked as a draft.
4. Open it to download the A4 PDF, print it, or e-mail it to the customer yourself.
5. Move it along as the conversation develops: **draft -> sent -> accepted / rejected / cancelled**. Quotations past their validity window are marked **expired**.

A quotation never touches stock and never takes money — it is a priced offer. Two settings control it: validity in days (14 by default) and the terms paragraph printed at the foot of the PDF.

3.4 Receipts: paper, PDF, or both

- **Thermal (default)** — laid out for an 80 mm receipt printer. Use the browser's print dialogue and choose the receipt printer.
- **A4** — add `?size=a4` to the receipt address, or pick A4 in the receipt page, for an office printer or a PDF by e-mail.
- **Save as PDF** — every receipt and quotation is a real PDF generated by the system, so “Save as PDF” in the print dialogue always produces a clean file.
- **Reprint** — *Sales* lists every sale; open one to reprint or download it. A cashier sees only their own sales, the admin sees all.

Receipt numbers are sequential per month (`RCP-202610-0007`) and are never reused, even when a sale is voided — a voided sale keeps its number and is marked **voided** in the sales list.

3.5 Wrong sale? Void it — don't delete it

The admin can void a completed sale from the receipt page, with a reason. Voiding puts the stock back on the shelf, records the refund in the cash ledger and keeps the audit trail. A sale in a month that has already been closed cannot be voided — that month's figures are locked.

3.6 Out of stock: capture the customer, then tell them when it lands

1. When an item shows **0 in stock**, the till offers *Not in stock? Take a customer's e-mail*.
2. Enter the e-mail (name, phone and a note are optional). The customer joins that part's waiting list.
3. The admin sees every waiting customer under *Back-in-stock*, and is also warned on the restock screen.
4. When stock arrives the system can e-mail the waiting customers automatically — that requires the shop's e-mail account to be configured once in Settings (SMTP). Without it, notify each customer with one click from that page.

> **Set up e-mail once and it runs itself**
> Settings -> Outgoing e-mail: server, port (587), username, password, from-address, TLS on. Use *Send test e-mail* to prove it works. With that in place, restocking a part e-mails everyone on its waiting list; without it you simply press *Notify* for each customer when you are ready.

4. Stock, purchasing and what the shop is worth

4.1 Products

Every part has a SKU, name, category, brand, compatible bike models, **selling price**, **cost price** (admin only), stock on hand, reorder level, an optional note and an active/archived flag. Archive rather than delete: archived parts vanish from the till but keep their history, so old receipts still make sense.

4.2 Stock levels and reorder warnings

- **Stock levels** shows every part with its quantity, reorder level and a flag when the quantity is at or below the reorder level, plus total units and the value of the shelf.
- **Low stock** appears on the dashboard, so the admin sees what to buy without hunting.
- **Stock ledger** is the complete, append-only history: opening stock, sales, restocks, adjustments, voids — each line with quantity, cost, resulting quantity, reference, user and timestamp. Nothing is edited or deleted in the ledger; that is what makes stock disputes answerable.

4.3 Restocking, and paying for it out of the business

1. Stock -> **Restock**: choose the part, quantity, the **unit cost you actually paid**, supplier and a note.
2. Tick **Pay from business cash** (and the method: cash, bank or mobile money) if the money came out of the till or the shop account. The expense is logged automatically so the cash balance stays honest.

3. Save. On-hand quantity rises, the average cost is recalculated, the ledger records the movement, and any waiting customers are queued for an e-mail.

The system keeps a **moving average cost**: buy 10 units at 200.00 and 5 at 240.00 and the cost used for profit becomes 213.33 per unit. That is why a restock at a new price quietly corrects the profit figures of the next sale — and why the unit cost must be typed honestly rather than guessed.

4.4 Adjustments: breakage, theft, stock-take differences

Stock -> **Adjust** takes a signed quantity and a reason (“damaged in store”, “count correction”). It never silently changes a price and it always appears in the ledger, so a shrinkage pattern is visible instead of hidden.

4.5 Breakages, returns and stock requests

A cashier cannot change stock, but they are the person who sees the damage or takes the return. So the shop runs on **requests**: anyone raises the change, the admin approves it, and only then does the shelf move.

Request type	What it does on approval	Typical use
Breakage / write-off	Takes the quantity out of stock and records the loss at cost	A part dropped while fitting, a damaged tyre, a cracked mirror
Customer return	Puts the quantity back into stock and, if you enter an amount, pays that refund out of business cash	Wrong size bought yesterday, warranty replacement
Stock correction	Adds or removes a quantity	Stock-take found three more or two fewer than the system

1. Open **Stock requests** in the sidebar (cashiers and admins both see it).
2. Choose the part, what happened, the quantity and a reason — the reason is mandatory and is copied onto the stock ledger.
3. Press **Send for approval**. The request waits as **pending**; stock is untouched.
4. The admin opens the same page and presses **Approve** (optionally adding a refund and a note) or **Refuse**. The pending count also appears on the dashboard and in the sidebar badge.
5. Approval writes the ledger line, updates the quantity and, for a return with a refund, posts the money out of the cash ledger.

> **Why ask instead of allowing it**
> Every approved breakage is a cost the owner can see in the monthly report, and a refused request keeps its reason and the admin's note on file. That is what stops stock quietly disappearing between counts.

4.6 What the stock is worth

Figure	Where to find it	What it means
Stock at cost	Dashboard / Stock levels	Money tied up in the shelf, valued at cost price.
Stock at retail	Stock levels	What the shelf would bring in at today's prices.
Potential margin	Stock levels	Retail value minus cost value.
Units and SKUs	Stock levels	How many pieces, how many different parts.

5. Money, profit and the monthly close

5.1 Business cash

Business cash is the running record of money moving in and out of the business that is not a sale. Every entry needs a kind, an amount, a method (cash, bank, mobile money) and a note; entries can be corrected by an adjustment with a reason.

Kind	Direction	Use it for
Operating expense	Money out	Rent, fuel, workshop consumables, airtime.
Salary paid	Money out	Wages paid to a team member from the Salaries page (automatic).
Owner withdrawal	Money out	Money the owner takes out of the business.
Capital injection	Money in	Money the owner puts in.
Adjustment	Money in	Corrections after a physical cash count.
Sale income	Money in	Created automatically by every completed sale.
Refund (voided sale)	Money out	Created automatically when a sale is voided.
Customer return refund	Money out	Created automatically when a return with a refund is approved.

The balance shown is money in minus money out, overall and per method. That single number is the answer to “how much money is in the business?” — and because sales and restock payments post automatically, it only stays true if every manual payment is logged as it happens.

5.2 How profit is calculated

- **Revenue** — what the customer paid, after discounts, including tax.
- **Cost of goods sold (COGS)** — the average cost of the units on that receipt, captured at the moment of sale.
- **Gross profit** — revenue (excluding tax) minus COGS. Shown per sale, per part, per category, per cashier and for any date range.
- **Net profit** — gross profit minus the operating expenses logged against the period, **salaries included**.
- **Discounts** are reported separately, so the owner can see exactly what discounting cost, who approved it and why.
- **Breakages** are shown at cost as their own figure, and **returns** show how many units came back — both kept out of the profit arithmetic so the numbers stay easy to explain.

5.3 Paying salaries

1. Open **Salaries**. The page shows the month's total paid, the current business cash balance and every payment already made.
2. Choose the person, the amount, the month it is for, whether it was paid by cash, bank or mobile money, and an optional note.
3. Press **Pay from business cash**. The money leaves the cash ledger immediately as *Salary paid* (reference such as `SAL-202610-002`), counts as an expense in the monthly close, and appears on that person's own Account page.

4. A salary payment cannot be deleted from the cash ledger; if one is wrong, pay a correcting adjustment and note why.

5.4 The monthly close

1. **Monthly close -> Close this month.** The system freezes that month's figures into a closing record: sales count, revenue, discounts, tax, COGS, gross profit, expenses, net profit, purchases paid from cash, voided sales and the best-selling parts.
2. The month becomes read-only for sales corrections: receipts can still be printed, but sales in that month can no longer be voided.
3. Every closing stays on file as a PDF-ready report (open it from *Monthly close*) for the accountant or the bank.

Close the month before you file the cash away
Closing is what turns daily numbers into a monthly record that cannot drift. Do it on the first working day after month end: open *Monthly close*, check revenue, COGS, gross and net against the drawer and the bank, then close and print the report.

5.5 Reports and exports

Report	Answers
Sales in a date range	Every receipt: number, date, cashier, customer, subtotal, discount, tax, total, profit, status.
Top performing parts	Which parts earn the most profit and the most revenue in the period.
Category performance	Where the money comes from by part category.
By cashier	Who sold how much, and how much they discounted.
Discount report	Every request: who asked, who approved, amount, reason, status.
Stock valuation	Units and value at cost and at retail, with low-stock flags.
Cash movement	Money in and out by kind and method, with the running balance.

CSV exports (sales, products, stock, discounts) open directly in Excel or Google Sheets — useful for the accountant, an insurer or a stock-take reconciliation.

6. Admin settings and the audit trail

6.1 Settings worth reviewing once

Setting	Why it matters
Business name, tagline, address, phone, e-mail	Printed on every receipt and quotation — it is the shop's stationery.
Currency symbol	Used throughout the app and on PDFs.
Tax rate and label (VAT / none)	Set to 0 until the business is VAT-registered; then the tax line appears on receipts and in the profit reports.
Maximum discount %	The ceiling a cashier or admin may request (default 60%).
Low-stock threshold	The default reorder level used for new parts.
Receipt footer	The thank-you line at the bottom of the paper receipt.
Quotation validity (days) and terms	Printed on every quotation PDF.
Outgoing e-mail (SMTP)	Powers the automatic back-in-stock e-mails.

6.2 Team & access

- Create a user with a role (**cashier** or **admin**), a full name and a starting password — one login per person, never a shared one.
- Reset a forgotten password, or deactivate a user who has left. Deactivated users cannot sign in but their sales stay in the records.
- **Delete** removes the account completely. If the person already has sales, quotations, discount decisions or salary payments, those rows are kept and the account is anonymised instead, so old receipts and payroll still make sense. A deleted user cannot sign in and disappears from the team list.
- Passwords are stored only as scrypt hashes. Nobody — including the technical team — can read an existing password; the only options are reset or deactivate.

6.3 Activity log

The audit trail records sign-ins and failed sign-ins, sales and voids, restocks and adjustments, discount requests and decisions, cash entries, settings changes and user changes — with the user, the timestamp, the affected record and the IP address. When a figure looks wrong, this is the first place to look.

6.4 Daily, weekly, monthly

When	Do this
Start of day	Sign in, check the dashboard for low stock and pending discounts, confirm the drawer float.
Every sale	Search, add, discount (if authorised), take payment, hand over the receipt.
When stock lands	Restock with the real unit cost; tick *pay from business cash* if you paid from the business; notify waiting customers.
End of day	Compare the drawer with *Business cash* and log any expense or withdrawal you made.
Weekly	Review low stock, send quotations that are going cold, look at the discount report.
Monthly	Close the month, print the closing report, export the CSVs for the accountant, back up the database (see 2.9).

7. Technical overview

The application is a single Python web service with server-rendered HTML. There is no JavaScript framework, no build step and no separate API tier, which is why it starts in a second and stays cheap to run.

Layer	Technology
Web framework	FastAPI (Python 3.12 on the server) with Uvicorn
Pages	Jinja2 templates, one CSS file and two small JavaScript files (no bundler)
Business logic	services.py — sales, stock ledger, discounts, cash, closings, analytics
PDF	ReportLab (80 mm thermal + A4 receipts, A4 quotations)
Database (cloud)	PostgreSQL 18 on Neon, accessed with psycopg 3 and a psycopg_pool connection pool
Database (local)	SQLite in WAL mode — the offline fallback on the shop PC
Hosting	Render free web service, Frankfurt (EU Central)
DNS / TLS	Cloudflare DNS, certificates issued and renewed by Render
CI / ops	GitHub repository with auto-deploy on `main` and a 10-minute keep-alive Action

7.1 Where things live

Environment	Address	Notes
Production	https://www.teamriders.bid	Primary address. Apex and `shop.` redirect to `www`.
Render origin	https://teamriders-pos.onrender.com	The service itself; works even if DNS is changed.
Source code	https://github.com/earnestchitsora92-dev/teamriders-pos	Public repository, `main` branch is production.
Cloud database	Neon project **teamriders**, database **neondb**	Connection string exists only in Render's environment variables.
Offline fallback	`http://127.0.0.1:8077` on the shop PC	SQLite file `data/mxspares.db`; Cloudflare tunnel `teamriders` can publish it again.

At handover (21 October 2026) the cloud database held the shop catalogue of 27 parts, two existing sales, one closed month and two user accounts (one admin, one cashier). Anything higher than that means the shop has been trading — which is the point.

7.2 How a request flows



7.3 Repository layout

Path	Purpose
app/main.py	App creation, session + CSRF middleware, login, setup, dashboard, health check
app/routes_sales.py	POS, sales, receipts and PDFs, quotations, discount requests and decisions
app/routes_admin.py	Products, stock, alerts, cash, closings, reports/CSV, settings, users, audit
app/services.py	All business rules and the SQL that implements them
app/db.py	Schema, backend switch (SQLite / PostgreSQL), settings cache, seed catalogue, money helpers
app/security.py	scrypt hashing, session tokens, login throttling, admin re-authentication
app/pdf.py	ReportLab receipts (thermal/A4) and quotation PDFs
app/templates, app/static	Jinja2 pages, app.css, app.js, pos.js
scripts/	migrate_to_postgres.py, push_to_github.py, deploy_render.py, pg_smoke_test.py, build_handbook.py
.github/workflows/keepalive.yml	Pings /healthz every 10 minutes so the free instance never sleeps
render.yaml	Blueprint equivalent of the live service (build, start, env vars, health check)
data/	SQLite database for the offline fallback (git-ignored)

8. Database, data model and integrity rules

The same schema is created for both backends; only the primary-key syntax differs (`SERIAL` in PostgreSQL, `INTEGER PRIMARY KEY AUTOINCREMENT` in SQLite).

8.1 Tables

Table	Holds
users	Accounts: username, full name, role (admin/cashier), scrypt hash, active flag, last sign-in
sessions	One row per signed-in device: token hash, CSRF secret, expiry, IP, user agent
products	Part master data: SKU, name, category, brand, compatible models, price, cost, stock, reorder level, active flag
stock_movements	Append-only ledger: kind (initial/sale/restock/adjustment/void), quantity delta, unit cost, resulting quantity, reference, user
discounts	Every request: code, kind, value, subtotal, amount, reason, requested_by, approved_by, status, expiry, sale_id
sales	Receipt header: number, cashier, customer, subtotal, discount, tax, total, COGS, profit, payment method, tendered, change, status, void details
sale_items	Receipt lines: product snapshot (SKU, name), quantity, unit price, unit cost, line total
quotations	Quote header: number, customer, totals, validity, status, created_by
quotation_items	Quote lines (same shape as sale items)
stock_alerts	Back-in-stock waiting list: product, customer e-mail/name/phone, note, status, notified_at
cash_transactions	Money in/out: kind, amount, method, reference, note, user, timestamp
payroll	Salary payments: person, month, amount, method, reference (SAL-YYYYMM-NNN), who paid it
stock_requests	Breakage / return / correction requests awaiting or carrying an admin decision, with the reason, direction, refund and decision note
closings	One row per closed month: the frozen figures plus a JSON snapshot and notes
sequences	Counters behind receipt and quotation numbers (per month)
settings	Key/value business configuration (see Appendix B)
audit_log	Who did what, when, from which IP

8.2 Non-negotiable rules in the code

- **Money is integer cents** everywhere (`price_cents`, `total_cents`, …). No floats, no rounding drift; percentages are rounded half-up.
- **Stock only moves through the ledger.** Selling, voiding, restocking and adjusting all write a `stock_movements` row and update the product quantity in the same transaction.
- **Costs use a moving average** — a restock blends the new unit cost into the existing average, which is what makes per-sale profit honest.
- **Receipt and quotation numbers** come from the `sequences` table inside the same transaction, so two cashiers can never take the same number.
- **A closed month is frozen**: `void_sale` refuses any receipt whose month has a closing record.
- **Stock changes need a decision.** A request only touches the shelf when an admin approves it; the ledger line, the new quantity and any refund are written in one transaction.
- **Salary payments are cash events.** Inserting a payroll row and its `cash_transactions` entry happens together, so the balance can never drift from the payment list.

- **Users with history are anonymised, not erased.** Deleting keeps the row (and every receipt it authorises) while scrambling the password and hiding the account.
- **Deletes are avoided.** Products are archived, sales are voided, cash entries are corrected by an adjustment — the history remains complete.

8.3 How one backend flag changes everything

At import time `app/db.py` reads `DATABASE_URL`. If it is set (and looks like a PostgreSQL URL) the app talks to PostgreSQL; otherwise it opens the SQLite file. Because the entire codebase is written in SQLite's dialect, the PostgreSQL path is provided by a thin adapter:

Difference	How it is handled
Placeholders	`?` is translated to `%s` on the way in.
Insert IDs	`RETURNING id` is appended to inserts into ID tables so `cursor.lastrowid` keeps working.
Row access	Rows are dicts that also allow sqlite3-style positional access (`row[0]`).
Numerics	PostgreSQL returns `Decimal` for aggregates; these are normalised to int/float so JSON and templates behave identically.
Dialect	Grouped columns are fully qualified and alias-based ORDER BY expressions were rewritten as sub-queries.
Connections	A psycopg pool (min 1, max 6, 15 s timeout) with per-request checkout/return and rollback.

9. Deployment, configuration and operations

9.1 Configuration (environment variables)

Variable	Value in production	Purpose
DATABASE_URL	`postgresql://…neon.tech/neondb?sslmode=require`	Switches the app to PostgreSQL. Set only in Render (Secret).
PYTHON_VERSION	3.12.7	Interpreter Render builds with.
PYTHONUNBUFFERED	1	Live logs.
MX_SQLITE_JOURNAL	delete	Ignored when PostgreSQL is used; kept for the SQLite fallback.
PORT	injected by Render	Uvicorn must bind `0.0.0.0:$PORT`.

Build command `pip install --upgrade pip && pip install -r requirements.txt`; start command `uvicorn app.main:app --host 0.0.0.0 --port $PORT --workers 1 --proxy-headers --forwarded-allow-ips '*'`; health check `/healthz`.

9.2 The change process

1. Edit the code and test locally (`python run.py`, then `python scripts/pg_smoke_test.py`).
2. Publish with `python scripts/push_to_github.py --repo teamriders-pos --message "…"` (no git install needed; the script uploads the tree through the GitHub API).

3. Render pulls the new commit and rebuilds. Because the service reads a *public repository URL* rather than a connected Git provider, the push alone does not start a build: trigger it with `python scripts/deploy_render.py --repo-url <repo>` (it merges the existing environment variables, so DATABASE_URL is kept) or press *Manual Deploy* in the dashboard.
4. Verify: `curl https://www.teamriders.bid/healthz` and sign in.
5. Roll back by choosing the previous successful deploy in Render and clicking *Redeploy*.

9.3 Handy scripts

Script	What it does
scripts/push_to_github.py	Creates/updates the GitHub repository from the working tree via the API.
scripts/deploy_render.py	Creates or updates the Render service, merges env vars, attaches domains and triggers a deploy (needs RENDER_API_KEY).
scripts/migrate_to_postgres.py	Copies a SQLite database into PostgreSQL, preserving IDs and resetting sequences; skips sessions.
scripts/pg_smoke_test.py	55 service-layer checks (auth, products, stock, discounts, sales, quotations, cash, closings, reports, alerts, stock requests, salaries, deletions) against whichever backend DATABASE_URL selects.
scripts/http_smoke_test.py	38 HTTP checks against a running instance: setup, logins, cashier restrictions, stock approval, salary payment, user and product deletion.
scripts/build_handbook.py	Regenerates this PDF from the current source tree.

9.4 Backups and restore

What	How	How often
Cloud database	Neon console -> *Backups / Restore* (point-in-time per branch), or `pg_dump "$DATABASE_URL" -Fc > teamriders.dump`	Weekly dump; Neon's own history covers the rest
Offline SQLite copy	Copy `data/mxspares.db` while the PC app is stopped	Whenever it is used in earnest
Accountant's figures	Reports -> CSV exports (sales, products, stock, discounts) and the monthly closing PDF	Monthly, with the close
Source code	GitHub repository (public) — every commit is a restorable version	Automatic

Restoring PostgreSQL
Create an empty database, run `pg_restore -d "$DATABASE_URL" --no-owner teamriders.dump`, then point Render's DATABASE_URL at it. The schema is created automatically by the app on first start, so restoring data alone is enough if you have a plain SQL dump.

9.5 Testing before and after a change



The suite creates its own users, products, discounts, sales, quotation, cash entry and closing, so run it against a scratch database (or accept the extra test rows). Never point it at the live database in business hours without telling the shop.

9.6 Keeping the free instance awake

Render's free plan stops an idle service after about 15 minutes; the next visitor then waits 30–60 seconds. `.github/workflows/keepalive.yml` requests `https://www.teamriders.bid/healthz` every 10 minutes, which keeps it warm. Check it under the repository's *Actions* tab; it can also be run manually (*Run workflow*). Any uptime monitor pointed at the same URL is an equivalent safety net.

10. Security, secrets and limits

10.1 What protects the shop

- **Passwords** are hashed with scrypt and a per-user salt; the plain text is never stored and cannot be recovered.
- **Sessions** are database-backed: the cookie carries a random token, only its hash is stored, it expires after 12 hours of inactivity, and it is invalidated on sign-out or password reset.
- **CSRF** — every form and JSON call carries a per-session token; a mismatch is rejected with 403.
- **Roles** are enforced on the server for every admin route, not hidden in the interface.
- **Transport** — HTTPS everywhere; cookies are marked Secure and HttpOnly behind the proxy, plus HSTS.
- **Throttling** — repeated failed sign-ins lock the username briefly and are written to the activity log.
- **Audit** — every state change is attributed to a user, a time and an IP address.
- **No card data** is captured or stored; card payments are taken on the bank's own device.

10.2 Secrets inventory

Secret	Where it lives	Rotation
Neon database URL	Render -> service -> Environment (Secret). Never in the repository	Neon -> Reset password, then update DATABASE_URL and redeploy
Render API key	Developer's password manager	Render -> Account settings -> API keys -> revoke and create
Cloudflare API token (DNS edit)	Developer's password manager	Cloudflare -> My Profile -> API Tokens -> Roll
GitHub token (repo + workflow)	Developer's password manager	GitHub -> Settings -> Developer settings -> regenerate
Shop SMTP password	Settings screen in the app (database)	Change at the mail provider, then re-enter in Settings
User passwords	Not recoverable (scrypt hashes)	Admin resets them from Team & access

Housekeeping note
During the handover the connection strings and API keys were placed in plain text files in the owner's profile folder (`neon.txt, render.txt, cloudflare.txt, ghtoken.txt`). Move them into a password manager and delete the files; anyone with those files controls the database, the hosting account and DNS.

10.3 Free-tier ceilings

Service	Free allowance	What happens at the limit
Render web service	512 MB RAM, 0.1 CPU, sleeps after ~15 min idle	Slow first request (the keep-alive Action prevents it); upgrade to 2 GB for $7/month if the shop grows
Neon PostgreSQL	0.5 GB storage, autosuspend when idle	Writes fail once full — export CSVs and prune old audit rows, or upgrade the plan
GitHub Actions	Unlimited minutes for public repositories	Scheduled runs may be delayed a few minutes
Cloudflare DNS	Free plan	No practical limit for this use

10.4 Known limitations

- Single currency and single shop; the same install cannot serve two branches.
- No barcode scanner input — search by typing SKU, name, brand or bike model.
- No offline sync: the PC fallback and the cloud database are separate data sets.
- No purchase orders or supplier invoices — restocking records quantity and unit cost only.
- Receipts are commercial documents, not fiscalised tax invoices; add the shop's VAT number in Settings if the business registers.
- E-mail notifications need an SMTP account; without one the waiting list is notify-by-hand.

11. Troubleshooting

Symptom	Likely cause	Fix
First page load takes ~40 s	Render free instance was asleep	Confirm the keep-alive Action is enabled and green under the repo's Actions tab
Site shows “Application failed to respond”	Service crashed or is rebuilding	Render -> Logs; check the latest deploy; redeploy the previous commit if the build is broken
Sign-in page loads but login fails	Wrong password, or username locked after failed attempts	Wait for the cool-off, or admin resets the password
Everything redirects to /setup	The app can see no users — usually a wrong DATABASE_URL	Check DATABASE_URL on Render; a fresh database will show the setup screen
“Forbidden” on a page a cashier used before	Role is cashier	Sign in as an admin for that screen
“Invalid or missing security token”	Page was left open too long; session expired	Reload the page and try again
“Not enough stock for …”	Quantity on hand is lower than the cart	Restock or adjust, or sell fewer units
“Discount … is pending and cannot be used”	It has not been approved yet	Admin approves it in Discounts, then sell again
“Discount approval has expired”	Approved discount is older than 30 minutes	Request a new one
“Period YYYY-MM is closed”	The month has been closed	Void is impossible by design; correct it as an adjustment
Receipt prints on two pages	Wrong paper size in the print dialogue	Choose the 80 mm receipt printer, or use the A4 version
Back-in-stock e-mails not arriving	SMTP not configured, or wrong credentials	Settings -> Outgoing e-mail -> Send test e-mail
Apex domain does not resolve	Stale negative cache from an earlier move	Prefer `www.`; Flush the local DNS cache or use what the domain shows
Site works on the Render URL but not the domain	Cloudflare record points at the old tunnel	Set the three CNAMEs to `teamriders-pos.onrender.com`, proxy off

12. The offline fallback on the shop PC

The PC copy exists for one reason: to keep selling when the internet or the cloud is unavailable. It runs the same code on SQLite.

1. Double-click `start.bat` in the project folder (or run `python run.py`).
2. Open `http://127.0.0.1:8077` on the PC, or `http://<pc-ip>:8077` from another device on the shop Wi-Fi.
3. Sign in with the local accounts — these are separate from the cloud accounts.
4. To publish the fallback through the old tunnel, run `start-online.bat` and point the Cloudflare CNAMEs back at `5bfc9160-5e58-4aa8-ae7a-4d19316b78a1.cfargotunnel.com` (proxy on).
5. When the cloud is back, either keep using one of them consistently, or copy the offline sales across deliberately (export the CSVs and re-enter them) — there is no automatic merge.

Rule of one till
Never sell from the cloud and the offline copy in the same period. Stock, receipts and profit are kept separately in each, and reconciling two sets of receipts by hand is exactly the work this system exists to avoid.

12.1 The Windows startup shortcut

A shortcut named *Team Riders POS* in the Startup folder launches the local app (and previously the tunnel) when the PC signs in. While the shop runs in the cloud this is unnecessary; remove it from `shell:startup` if you want the PC to stay clean, and start the fallback by hand only when needed.

Appendix A — Route reference

Path	Role	Purpose
/healthz	public	Liveness probe used by Render and the keep-alive Action
/login, /logout, /setup	public	Authentication; setup only exists while there are no users
/dashboard	any	Today/this month figures, low stock, attention list, latest receipts
/pos	any	The till: search, cart, discount, payment, quote mode
/api/products/search	any	Type-ahead suggestions with price and stock
/api/discounts, /api/discounts/{code}	any	Create a discount request / look one up
/api/sales, /api/quotations, /api/alerts	any	Create a sale, a quotation, or a back-in-stock alert
/sales, /sales/{id}	own / all	Sales list and receipt detail
/sales/{id}/receipt, /sales/{id}/receipt.pdf?size=a4	own / all	Printable receipt, thermal or A4
/sales/{id}/void	admin	Void a completed sale (blocked for closed months)
/quotations, /quotations/{id}, /quotations/{id}.pdf	any	Quotation list, detail, PDF
/discounts, /discounts/{id}/decide	admin	Approve or refuse discount requests
/products, /products/new, /products/{id}/edit, /products/{id}/toggle	admin	Catalogue maintenance
/products/{id}/delete	admin	Delete a part (archived instead when it has sales history)
/stock, /stock/movements, /stock/restock, /stock/adjust	admin	Levels, ledger, restock, adjustment
/stock/requests	any	Raise a breakage, return or correction request; admin approves or refuses
/salaries, /salaries/pay	admin	Pay a salary from business cash and see the payment history
/alerts, /alerts/{id}/notify, /alerts/notify-ready	admin	Back-in-stock waiting list
/cash, /cash/add, /cash/{id}/delete	admin	Business cash ledger
/closings, /closings/close, /closings/{id}	admin	Monthly close-off and its report
/reports, /reports/export/{sales|products|stock|discounts}.csv	admin	Period reporting and exports
/settings, /settings/test-email	admin	Business configuration and SMTP test
/users, /users/new, /users/{id}/password, /users/{id}/toggle, /users/{id}/delete	admin	Team & access, including deleting an account
/audit	admin	Activity log
/account, /account/password	any	Own profile and password

Appendix B — Settings keys

Key	Default	Meaning
business_name	Team Riders	Shop name on screens and documents

Key	Default	Meaning
business_tagline / address / phone / email	—	Stationery details on receipts and quotations
currency	$	Symbol used everywhere
tax_rate_percent / tax_label	0 / VAT	Tax rate and the wording of the tax line
max_discount_percent	60	Ceiling for any discount request
low_stock_threshold	5	Default reorder level for new products
receipt_footer	Thank you for your business…	Line printed at the foot of receipts
quotation_valid_days / quotation_terms	14 / —	Validity window and terms on quotations
smtp_host / port / user / password / from / tls	blank / 587 / …	Outgoing e-mail for back-in-stock notices
business_initialised	0 -> 1	Set once the owner account exists; hides the setup screen

Appendix C — Enumerations used by the data

Field	Allowed values
users.role	admin, cashier
sales.status	completed, voided
sales.payment_method	cash, mobile, card, bank
stock_movements.kind	initial, sale, restock, adjustment, void, breakage, return
discounts.kind	percent, amount
discounts.status	pending, approved, denied, expired, used
quotations.status	draft, sent, accepted, rejected, cancelled, expired
stock_alerts.status	waiting, notified, cancelled
cash_transactions.kind	expense, owner_withdrawal, capital_injection, adjustment, sale_income, refund, salary, customer_refund
stock_requests.kind	breakage, return, correction
stock_requests.status	pending, approved, denied
users.is_deleted	0 active account, 1 deleted (anonymised, cannot sign in)
cash_transactions.method	cash, bank, mobile

Document generated from the source tree by `scripts/build_handbook.py`. Regenerate it whenever a screen, setting or deployment detail changes, so the handbook never drifts from the system.